

Assignment 2 – Testing Techniques

Author: Ma Thanh Long Mason

Student ID: S4042629

Table of Contents

Table of Figures	3
Section 1: Intrusion Detection.....	4
Q1.1	4
Q1.2	4
Q1.3	4
Q1.4	4
Q1.5	4
Q1.6	5
Q1.7	5
Q1.8	5
Q1.9	6
Section 2: Firewalls	7
Q2.1	7
Q2.2	7
Q2.3	8
Q2.4	9
Q2.5	9
Section 3: Proper Usage of Symmetric Encryption	10
Q3.1	10
Q3.2	11
Q3.3	12
Section 4: Penetration Testing	13
Q4.1	13
Q4.2	15
Q4.3	16
Q4.4	17
Q4.5	17
Q4.6	18

Table of Figures

Figure 1: Snort rule in terminal	5
Figure 2: Pinging 10.0.2.0 for Snort testing.....	5
Figure 3: Snort alerts for pinging 10.0.2.0	6
Figure 4: Pinging 2.6.2.9.....	6
Figure 5: Snort detecting pings outside of subnet 10.0.2.0/24	6
Figure 6: Preventing pinging to outside machines.....	7
Figure 7: Blocking replies from 8.8.8.8 via firewall.....	7
Figure 8: FORWARD table for Q2.3.....	8
Figure 9: FORWARD table for Q2.4.....	9
Figure 10: FORWARD table for Q2.5.....	9
Figure 11: Output encrypted files	10
Figure 12: ECB Encrypted file's hex	11
Figure 13: CBC Encrypted file's hex	11
Figure 14: Original image's hex	12
Figure 15: Images after hex edit	12
Figure 16: ECB encrypted image.....	12
Figure 17: CBC encrypted image	13
Figure 18: Operating system information of RMIT web server	15
Figure 19: Ports and Services of RMIT web server.....	16
Figure 20: Analysis of int_overflow_vuln.c	17
Figure 21: Revised vulnerable code - Part 1	18
Figure 22: Revised vulnerable code - Part 2	18

Section 1: Intrusion Detection

Q1.1

Chosen method: HIDS

Justification: Since HTTPS traffic is encrypted, NIDS won't be able to directly inspect and packets that are being sent. That's why the inspection must happen after the decryption of the packets which happens on the host side. Therefore, HIDS will be most suitable to immediately detect and flag any malicious packets.

Q1.2

Chosen method: Logging

Justification: Since we are working with HTTP, both NIDS and HIDS can be used for intrusion detection. However, since the requirement specifically asked for the use of a computationally expensive Machine Learning algorithm and low latency, logging would be most suitable for this case since both NIDS and HIDS would be too performance-taxing. Logging would allow us to store and analyse the traffic either offline or asynchronously, ensuring the analysis process wouldn't impact the performance of the web application.

Q1.3

Chosen method: NIDS

Justification: Since we are working with unencrypted traffic, NIDS would be the most suitable method for this case. We are trying to detect phishing attacks which require scanning and stopping payloads before they reach the endpoint, at which point it would be too late. HIDS is not possible since by the time the payloads have reached the host, the attack is already considered successfully; logging is simply too slow for this scenario.

Q1.4

Chosen technique: Signature-based detection

Justification: Since we are dealing with widely available tools and/or exploits, information about these methods is already well-known and extensively documented, thus can be easily identified by detecting the signature patterns that these methods give off.

Q1.5

Chosen technique: Specification-based detection

Justification: Since we are only trying to detect access to one specific file. Rules can be predefined to detect and prevent access to the “/etc/passwd” file, hence the chosen technique.

Q1.6

Chosen technique: Anomaly-based detection

Justification: Since custom exploits aren’t documented, signature-based detection will fail to detect an attack. We are dealing with unknown exploits, an anomaly-based detection method will be more suitable since we can establish a baseline for normal activity and flag anything else that deviates from that baseline.

Q1.7

```
alert icmp any any -> 10.0.2.0/24 any (msg:"Ma Thanh Long Mason external ICMP test detected against the subnet"; itype:8; sid:100001;)
```

Figure 1: Snort rule in terminal

Snort rule:

“alert icmp any any -> 10.0.2.0/24 any (msg:"Ma Thanh Long Mason external ICMP test detected against the subnet"; itype:8; sid:100001;)”

This is basically an alert set to detect any ICMP packets going into any hosts within the 10.0.2.0/24 subnet. The rule will give an alert with the message specified in the “msg” field. Furthermore, the “itype:8” field specifies the packet type; packets type 8 are “Echo Request” packets which are sent when you are pinging from one machine to another. Finally, the “sid” field is a basic identifier for rule, which specifies the ID of this rule.

Q1.8

Below is the output after setting the rule for Snort. Snort was able to successfully detect all pings within the 10.0.2.0/24 subnet.

```
[09/13/25]seed@VM:.../rules$ ping -b 10.0.2.0
WARNING: pinging broadcast address
PING 10.0.2.0 (10.0.2.0) 56(84) bytes of data.
^C
--- 10.0.2.0 ping statistics ---
9 packets transmitted, 0 received, 100% packet loss, time 8196ms
```

Figure 2: Pinging 10.0.2.0 for Snort testing

```
[09/13/25]seed@VM:~/rules$ sudo /usr/sbin/snort -A console -q -u snort -g snort -c /etc/snort/snort.conf -i enp0s3
09/13-03:59:54.767817  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-03:59:55.795561  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-03:59:56.821141  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-03:59:57.861808  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-03:59:58.867513  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-03:59:59.891282  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-04:00:00.915515  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-04:00:01.939421  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
09/13-04:00:02.963561  [**] [1:100001:0] Ma Thanh Long Mason external ICMP test detected against the subnet [**] [Priority: 0] {ICMP} 10.0.2.4 -> 10.0.2.0
```

Figure 3: Snort alerts for pinging 10.0.2.0

Q1.9

Command used:

“alert icmp any any -> !10.0.2.0/24 any (msg:"Ping outside of 10.0.2.0/24 detected "; itype:8; sid:100002;)”

The command above is used to create a rule to detect pings outside of the 10.0.2.0/24 subnet. This command is added purely for demonstration purposes and is necessary to showcase that the rule set in Q1.7 is effectively ignoring pings outside of the designated subnet.

```
[09/25/25]seed@VM:~/rules$ ping 2.6.2.9
PING 2.6.2.9 (2.6.2.9) 56(84) bytes of data.
^C
--- 2.6.2.9 ping statistics ---
25 packets transmitted, 0 received, 100% packet loss, time 24559ms
```

Figure 4: Pinging 2.6.2.9

```
09/25-08:09:34.970141  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:35.977821  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:37.003108  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:38.025391  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:39.050355  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:40.076588  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:41.097093  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:42.121346  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:43.145054  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:44.169398  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
09/25-08:09:45.193096  [**] [1:100002:0] Ping outside 10.0.2.0/24 detected [**] [Priority: 0] {ICMP} 10.0.2.4 -> 2.6.2.9
```

Figure 5: Snort detecting pings outside of subnet 10.0.2.0/24

Section 2: Firewalls

Q2.1

```
root@0c4627c17936:/# iptables -A OUTPUT -p icmp --icmp-type echo-request -d 2.6.2.9 -j DROP
root@0c4627c17936:/# iptables -L OUTPUT --line-numbers -n
Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination            icmpype 8
1    DROP          icmp -- 0.0.0.0/0              2.6.2.9
root@0c4627c17936:/# ping 2.6.2.9
PING 2.6.2.9 (2.6.2.9) 56(84) bytes of data.
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
^C
--- 2.6.2.9 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4078ms
```

Figure 6: Preventing pinging to outside machines

Rule:

“iptables -A OUTPUT -p icmp --icmp-type echo-request -d 2.6.2.9 -j DROP”

-A: Specify which table to append this command to

-p: Specify which packet type to target. ICMP is the packet commonly used to send pings

--icmp-type: Specify specific packet type of ICMP packets; echo-request are request packets sent when pinging from one device to another.

-d: Specify the destination

-j: Specify the action to perform when an appropriate target is identified

The command above adds a rule to the output table, which drops all ICMP packets from the router to the machine with IP 2.6.2.9. This rule essentially targets all out-going ICMP request packets to 2.6.2.9 and drop them before they could be sent.

Q2.2

```
root@0c4627c17936:/# iptables -L INPUT --line-numbers -n
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination            icmpype 0
1    DROP          icmp -- 8.8.8.8                0.0.0.0/0
root@0c4627c17936:/# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
9 packets transmitted, 0 received, 100% packet loss, time 8186ms
```

Figure 7: Blocking replies from 8.8.8.8 via firewall

Rule:

“iptables -A INPUT -p icmp --icmp-type echo-reply -s 8.8.8.8 -j DROP”

- A: Specify which table to append this command to
- p: Specify which packet type to target. ICMP is the packet commonly used to send pings
- icmp-type: Specify specific packet type of ICMP packets; echo-reply are reply packets sent after receiving a request packet from the source.
- s: Specify the source
- j: Specify the action to perform when an appropriate target is identified

The command above adds a rule to the input table, which drops all replies received from 8.8.8.8. Notice how the console doesn't show anything but still reports back saying 9 packets were sent, which indicates that the reply packets were successfully dropped.

Q2.3

```
root@0c4627c17936:/# iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 18 packets, 1180 bytes)
pkts bytes target    prot opt in     out     source               destination          tcp dpt:23
 43  2335 ACCEPT    tcp  --  *      *       192.168.60.0/24      0.0.0.0/0             tcp dpt:23
 37  2758 ACCEPT    tcp  --  *      *       0.0.0.0/0            192.168.60.0/24      tcp spt:23
 95  5144 ACCEPT    tcp  --  *      *       0.0.0.0/0            192.168.60.5         tcp dpt:23
 81  5500 ACCEPT    tcp  --  *      *       192.168.60.5        0.0.0.0/0             tcp spt:23
```

Figure 8: FORWARD table for Q2.3

Commands:

1. iptables -A FORWARD -p tcp -s 192.168.60.0/24 --dport 23 -j ACCEPT
2. iptables -A FORWARD -p tcp -d 192.168.60.0/24 --sport 23 -j ACCEPT
3. iptables -A FORWARD -p tcp -d 192.168.60.5 --dport 23 -j ACCEPT
4. iptables -A FORWARD -p tcp -s 192.168.60.5 --sport 23 -j ACCEPT
5. iptables -P FORWARD DROP

- A: Specify which table to append this command to
- p: Specify which packet type to target.
- s: Specify the source
- d: Specify the destination
- sport: Specify the source port
- dport: Specify the destination port
- j: Specify the action to perform when an appropriate target is identified
- P: Used to specify the default policy to use.

Commands 1 and 2 allow internal hosts to communicate using telnet sessions. Commands 3 and 4 allow outside hosts to only communicate via telnet sessions to 192.168.60.5 (host 1). Command 5 blocks all other packets not specified in the rules above.

Q2.4

```
root@0c4627c17936:/# iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 14 packets, 1070 bytes)
  pkts bytes target    prot opt in     out     source               destination          ctstate
  83   5023 ACCEPT    all  --  *      *       0.0.0.0/0            0.0.0.0/0            RELATED,ESTABLISHED
  1    60 ACCEPT    tcp  --  *      *       0.0.0.0/0            192.168.60.5         tcp dpt:23 ctstate NEW
  6    360 DROP      tcp  --  *      *       192.168.60.0/24      0.0.0.0/0            tcp dpt:23 ctstate NEW
```

Figure 9: FORWARD table for Q2.4

Commands:

1. `iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`
2. `iptables -A FORWARD -p tcp -d 192.168.60.5 --dport 23 -m conntrack --ctstate NEW -j ACCEPT`
3. `iptables -A FORWARD -p tcp -s 192.168.60.0/24 --dport 23 -m conntrack --ctstate NEW -j DROP`
4. `iptables -P FORWARD DROP`

-A: Specify which table to append this command to

-p: Specify which packet type to target.

-s: Specify the source

-d: Specify the destination

-sport: Specify the source port

-dport: Specify the destination port

-j: Specify the action to perform when an appropriate target is identified

-P: Used to specify the default policy to use.

-m: Tell iptables to use an extension module to match packets based on certain criteria

--ctstate: Specifies the state of the connection for a packet

Command 1 creates a rule which allows reply packets back through for communication during telnet sessions to succeed. Command 2 allows outside hosts to only telnet to 192.160.60.5 (host 1). Command 3 prevents all internal hosts from starting a telnet session to outside hosts. Finally, command 4 drops all other packets not accounted for during the first 3 commands.

Q2.5

```
root@0c462/c1/936:/# iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 13 packets, 986 bytes)
  pkts bytes target    prot opt in     out     source               destination          ctstate
  348 20339 ACCEPT    all  --  *      *       0.0.0.0/0            0.0.0.0/0            RELATED,ESTABLISHED
  2   120 ACCEPT    tcp  --  *      *       192.168.60.0/24      0.0.0.0/0            tcp dpt:23 ctstate NEW
  1    60 ACCEPT    _tcp --  *      *       0.0.0.0/0            192.168.60.5         tcp dpt:23 ctstate NEW
```

Figure 10: FORWARD table for Q2.5

Commands:

1. `iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`
2. `iptables -A FORWARD -p tcp -s 192.168.60.0/24 --dport 23 -m conntrack --ctstate NEW -j ACCEPT`

3. `iptables -A FORWARD -p tcp -d 192.168.60.5 --dport 23 -m conntrack --ctstate NEW -j ACCEPT`
4. `iptables -P FORWARD DROP`

-A: Specify which table to append this command to

-p: Specify which packet type to target.

-s: Specify the source

-d: Specify the destination

-sport: Specify the source port

-dport: Specify the destination port

-j: Specify the action to perform when an appropriate target is identified

-P: Used to specify the default policy to use.

-m: Tell iptables to use an extension module to match packets based on certain criteria

--ctstate: Specifies the state of the connection for a packet

Command 1 will allow reply packets to go through, thus allowing telnet sessions to successfully operate. Command 2 will allow internal hosts to telnet to outside hosts. Command 3 will allow only telnet sessions to the host with IP 192.168.60.5 (host 1) on the internal side. Finally, command 4 will drop all other packets not specified.

Section 3: Proper Usage of Symmetric Encryption

Q3.1

Commands used:

ECB: `"openssl enc -aes-128-ecb -in pic_original.bmp -out pic_encrypted_ecb.bmp -K aa44004422662299aa44004422662299 -nosalt:"`

CBC: `"openssl enc -aes-128-cbc -in pic_original.bmp -out pic_encrypted_cbc.bmp -K aa44004422662299aa44004422662299 -iv 0a04000402060209 -nosalt"`

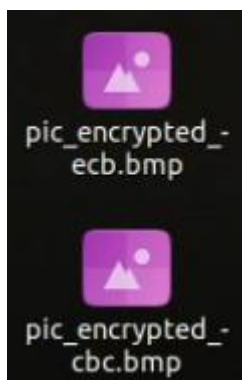


Figure 11: Output encrypted files

Q3.2

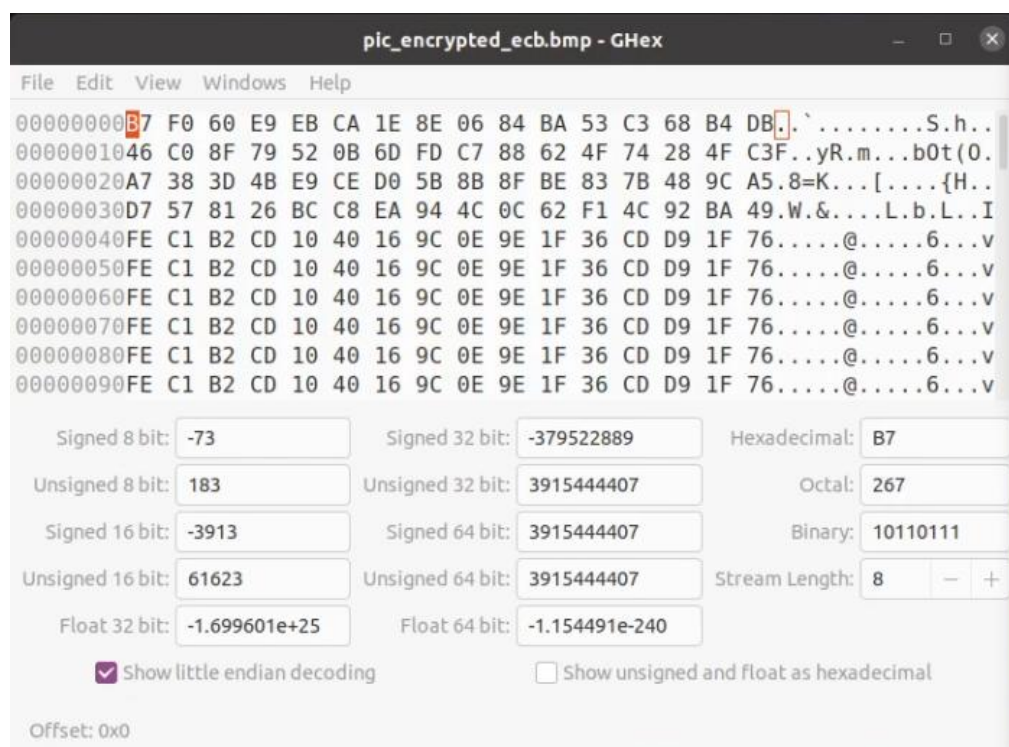


Figure 12: ECB Encrypted file's hex

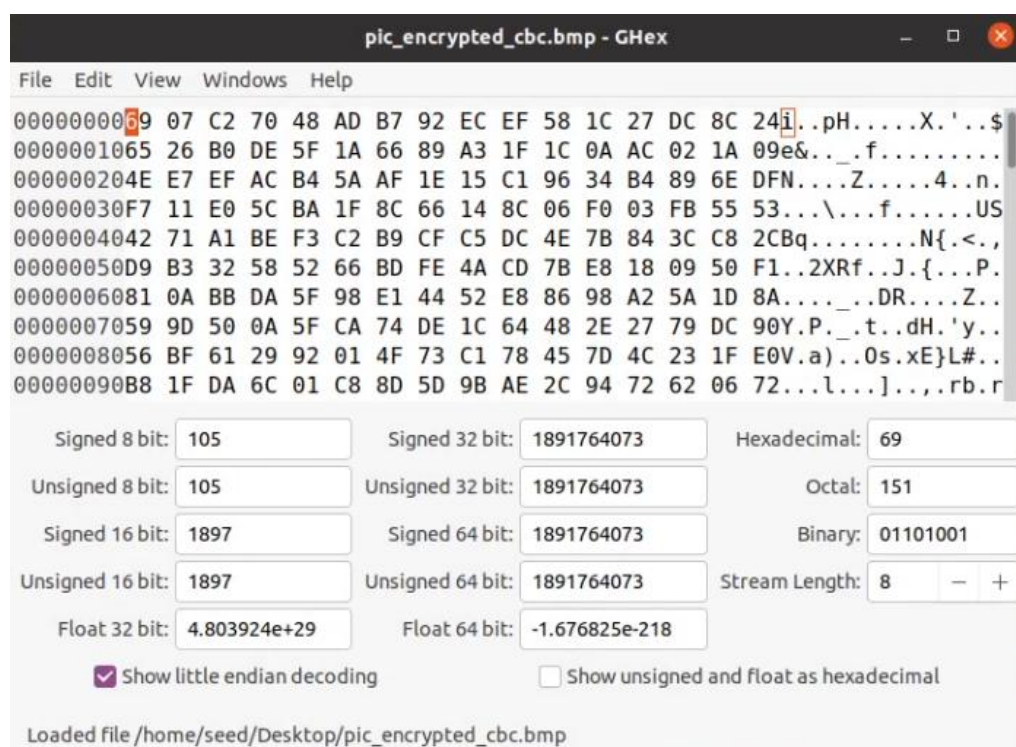


Figure 13: CBC Encrypted file's hex



Figure 14: Original image's hex

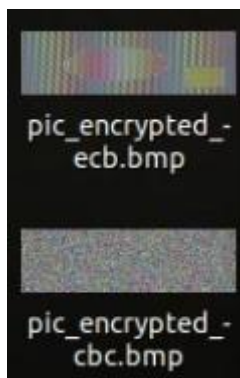


Figure 15: Images after hex edit

Q3.3

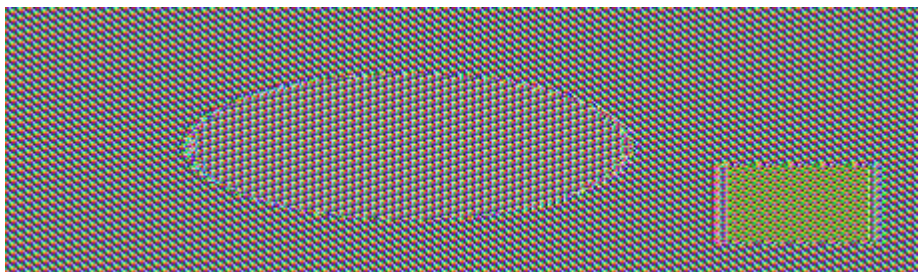


Figure 16: ECB encrypted image



Figure 17: CBC encrypted image

Observations:

Due to the way ECB works, which encrypts each text block independently, patterns can still be deprived from the encrypted image. Compared to the CBC, which carries information from encryption from one block to another, thus resulting in a more secure encrypted image. Specifically, in this case, both encryption mechanisms successfully hide the original color of the images. However, in the ECB image, the shapes shown in the original picture can still be identified.

Section 4: Penetration Testing

Q4.1

```
Starting Nmap 7.80 ( https://nmap.org ) at 2025-09-25 09:15 UTC
Nmap scan report for VM (192.168.60.1)
Host is up (0.000080s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
23/tcp    open  telnet   Linux telnetd
MAC Address: 02:42:42:F5:C6:C1 (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.80%E=4%D=9/25%OT=21%CT=1%CU=36814%PV=Y%DS=1%DC=D%G=Y%M=024242%T
OS:M=68D50843%P=x86_64-pc-linux-gnu)SEQ(SP=FD%GCD=1%ISR=100%TI=Z%CI=Z%II=I%
OS:TS=A)OPS(O1=M5B4ST11NW7%O2=M5B4ST11NW7%O3=M5B4NNT11NW7%O4=M5B4ST11NW7%O5
OS:=M5B4ST11NW7%O6=M5B4ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=
OS:FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%0=M5B4NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%
OS:A=S+F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%0=%RD=0
OS:%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+F=AR%0=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S
OS:=A%A=Z%F=R%0=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+F=AR%0=%RD=0%Q=)U1(R
OS:=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N
OS:%T=40%CD=S)

Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.08 ms   VM (192.168.60.1)
```

```
Nmap scan report for host1-192.168.60.5.net-192.168.60.0 (192.168.60.5)
Host is up (0.000019s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
23/tcp    open  telnet  Linux telnetd
MAC Address: 02:42:C0:A8:3C:05 (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.80%E=4%D=9/25%OT=23%CT=1%CU=35476%PV=Y%DS=1%DC=D%G=Y%M=0242C0%T
OS:M=68D50843%P=x86_64-pc-linux-gnu)SEQ(SP=104%GCD=1%ISR=107%TI=Z%CI=Z%II=I
OS:%TS=A)OPS(O1=M5B4ST11NW7%O2=M5B4ST11NW7%O3=M5B4NNT11NW7%O4=M5B4ST11NW7%O
OS:5=M5B4ST11NW7%O6=M5B4ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6
OS:=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0
OS:%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=
OS:0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%
OS:S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(
OS:R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=
OS:N%T=40%CD=S)

Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1    0.02 ms  host1-192.168.60.5.net-192.168.60.0 (192.168.60.5)
```

```
Nmap scan report for host2-192.168.60.6.net-192.168.60.0 (192.168.60.6)
Host is up (0.000014s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
23/tcp    open  telnet  Linux telnetd
MAC Address: 02:42:C0:A8:3C:06 (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.80%E=4%D=9/25%OT=23%CT=1%CU=42576%PV=Y%DS=1%DC=D%G=Y%M=0242C0%T
OS:M=68D50843%P=x86_64-pc-linux-gnu)SEQ(SP=107%GCD=1%ISR=10B%TI=Z%CI=Z%II=I
OS:%TS=A)OPS(O1=M5B4ST11NW7%O2=M5B4ST11NW7%O3=M5B4NNT11NW7%O4=M5B4ST11NW7%O
OS:5=M5B4ST11NW7%O6=M5B4ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6
OS:=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0
OS:%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=
OS:0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%
OS:S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(
OS:R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=
OS:N%T=40%CD=S)

Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1    0.01 ms  host2-192.168.60.6.net-192.168.60.0 (192.168.60.6)
```

```

Nmap scan report for host3-192.168.60.7.net-192.168.60.0 (192.168.60.7)
Host is up (0.000014s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
23/tcp    open  telnet  Linux telnetd
MAC Address: 02:42:C0:A8:3C:07 (Unknown)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.80%E=4%D=9/25%OT=23%CT=1%CU=33361%PV=Y%DS=1%DC=D%G=Y%M=0242C0%T
OS:M=68D50843%P=x86_64-pc-linux-gnu)SEQ(SP=105%GCD=1%ISR=108%TI=Z%CI=Z%II=I
OS:%TS=A)OPS(O1=M5B4ST11NW7%O2=M5B4ST11NW7%O3=M5B4NNT11NW7%O4=M5B4ST11NW7%O
OS:5=M5B4ST11NW7%O6=M5B4ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6
OS:=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%0=M5B4NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0
OS:A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%0=%RD=
OS:0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%0=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0
OS:S=A%A=Z%F=R%0=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%0=%RD=0%Q=)U1(
OS:R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=
OS:N%T=40%CD=S)

Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.01 ms  host3-192.168.60.7.net-192.168.60.0 (192.168.60.7)

Nmap scan report for 0c4627c17936 (192.168.60.11)
Host is up (0.000063s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
23/tcp    open  telnet  Linux telnetd
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.32
OS details: Linux 2.6.32
Network Distance: 0 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

Command used: “nmap -A 192.168.60.0/24”

The command above performs an aggressive nmap scan into the subnet “192.168.60.0/24”. The “-A” is used to tell nmap to perform an aggressive “bundle” scan, which retrieves the most basic information about the target such as the operating system, routes, ports, etc.

Q4.2

```

[09/25/25]seed@VM:~/Desktop$ sudo nmap -O rmit.edu.au
Starting Nmap 7.80 ( https://nmap.org ) at 2025-09-25 08:28 EDT
Nmap scan report for rmit.edu.au (131.170.0.105)
Host is up (0.012s latency).
Not shown: 995 filtered ports
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
443/tcp   open  https
554/tcp   open  rtsp
1723/tcp  open  pptp
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: bridge
Running: Oracle Virtualbox
OS CPE: cpe:/o:oracle:virtualbox
OS details: Oracle Virtualbox

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.95 seconds

```

Figure 18: Operating system information of RMIT web server

Command used: “sudo nmap -O rmit.edu.au”

The command above attempts to retrieve the basic operating information of RMIT's website. However, due to security measures set up by the target, nmap can't accurately retrieve OS information and the output shown in the terminal is purely an educated guess made by nmap.

Q4.3

```
Completed Service scan at 08:39, 11.27s elapsed (5 services on 1 host)
NSE: Script scanning 131.170.0.105.
Initiating NSE at 08:39
Completed NSE at 08:39, 0.01s elapsed
Initiating NSE at 08:39
Completed NSE at 08:39, 0.00s elapsed
Nmap scan report for rmit.edu.au (131.170.0.105)
Host is up (0.024s latency).
Not shown: 995 filtered ports
PORT      STATE SERVICE      VERSION
21/tcp    open  tcpwrapped
80/tcp    open  tcpwrapped
443/tcp   open  tcpwrapped
554/tcp   open  rtsp?
1723/tcp  open  tcpwrapped
```

Figure 19: Ports and Services of RMIT web server

Command(s) used: “sudo nmap -v -vS rmit.edu.au” or “sudo nmap -v -vS --version-light rmit.edu.au”

In this specific scenario, these two commands above give out the same output and are interchangeable. The reason the “—version-light” version was used was because the command is noticeably more lightweight than the full command, which means the runtime when much faster. Due to either the security measures of the RMIT website or latency, the first command takes a long time to run, thus the second command was used for demonstration in the video.

Q4.4

```
[09/19/25]seed@VM:~/.../flawfinder-1.27$ flawfinder int_overflow_vuln.c
Flawfinder version 1.27, (C) 2001-2004 David A. Wheeler.
Number of dangerous functions in C/C++ ruleset: 160
Examining int_overflow_vuln.c
int_overflow_vuln.c:7:  [2] (buffer) char:
    Statically-sized arrays can be overflowed. Perform bounds checking,
    use functions that limit length, or ensure that the size is larger than
    the maximum possible length.
int_overflow_vuln.c:14:  [2] (buffer) memcpy:
    Does not check for buffer overflows when copying to destination. Make
    sure destination can always hold the source data.
int_overflow_vuln.c:15:  [2] (buffer) memcpy:
    Does not check for buffer overflows when copying to destination. Make
    sure destination can always hold the source data.
int_overflow_vuln.c:26:  [1] (buffer) strlen:
    Does not handle strings that are not \0-terminated (it could cause a
    crash if unprotected).
int_overflow_vuln.c:27:  [1] (buffer) strlen:
    Does not handle strings that are not \0-terminated (it could cause a
    crash if unprotected).

Hits = 5
Lines analyzed = 32 in 0.50 seconds (11244 lines/second)
Physical Source Lines of Code (SLOC) = 23
Hits@level = [0]  0 [1]  2 [2]  3 [3]  0 [4]  0 [5]  0
Hits@level+ = [0+]  5 [1+]  5 [2+]  3 [3+]  0 [4+]  0 [5+]  0
Hits/KSLOC@level+ = [0+] 217.391 [1+] 217.391 [2+] 130.435 [3+]  0 [4+]  0 [5+]  0
Minimum risk level = 1
```

Figure 20: Analysis of `int_overflow_vuln.c`

Command used: “`flawfinder int_overflow_vuln.c`”

Observations:

Flawfinder was able to detect at least 160 dangerous functions within the code, along with 5 vulnerability hits on 5 different lines of code.

Q4.5

Vulnerability 1:

- Line: 7
- Severity: 2
- Explanation: A fixed-size array was found which could potentially overflow if the code writes more data into the array than it can handle. This could cause the data within the array to get corrupted in the event of an overflow. Furthermore, data overflow can potentially lead to system crashes.

Vulnerability 2, 3:

- Line(s): 14, 15
- Severity: 2

- Explanation: memcpy doesn't automatically check for buffer overflows during usage which could potentially happen when the destination isn't large enough to hold the data being transferred.

Vulnerability 4, 5:

- Line(s): 26, 27
- Severity: 1
- Explanation: strlen() expects a '\0' terminated string. If the source buffer doesn't include a null byte, like for instance when it was filled via memcpy without terminating, the function can read past the buffer, thus causing out-of-bounds reads and crashes.

Q4.6

```

1#include <stdlib.h>
2#include <stdio.h>
3#include <string.h>
4#include <limits.h> /* Function added for SIZE_MAX */
5
6int catvars(char *buf1, char *buf2, unsigned int len1, unsigned int len2)
7{
8    char mybuf[256];
9
10    /* Check and prevent int overflow */
11    if (len1 > SIZE_MAX - len2)
12    {
13        return -2;
14    }
15
16    /* Check for buffer overflow flawed -- arithmetic overflow! */
17    if ((len1 + len2) >= sizeof(mybuf)) /* Changed constant number of 256 to variable */
18    {
19        return -1;
20    }
21
22    memcpy(mybuf, buf1, len1);
23    memcpy(mybuf + len1, buf2, len2);
24    mybuf[len1 + len2] = '\0'; /* Ensure null-termination */
25
26    return 0;
27}
28
29
30int main(int argc, char **argv)
31{
32    int res;
33    size_t len1, len2; /* Changed from int to size_t */

```

Figure 21: Revised vulnerable code - Part 1

```

34
35    /* Sanitise the program's input */
36    if (argc < 3)
37    {
38        fprintf(stderr, "Usage: %s <str1> <str2>\n", argv[0]);
39        return 1;
40    }
41
42
43    len1 = strlen(argv[1]);
44    len2 = strlen(argv[2]);
45    res = catvars(argv[1], argv[2], len1, len2);
46
47    return res;
48}
49

```

Figure 22: Revised vulnerable code - Part 2

Changes:

- Added <limits.h> for SIZE_MAX which represents the maximum value which can be stored in a "size_t" variable. This variable is used to check for integer overflows.

- Changed the constant “256” to “sizeof(mybuf)” for better code readability and maintenance.
- Added “mybuf[len1 + len2] = ‘\0’;” which ensures a termination point for functions that work with strings like “strlen”. Since “memcpy” only copies raw bytes and doesn’t add the necessary “0” to indicate a string ending, we can manually add in the zero to prevent any overflows.
- Changed from “int” to “size_t”. “size_t” is a data type in C which is used to represent the size of objects in memory. This variable changes depending on the system’s architecture and automatically scales to the right word size for the machine. We can utilize this variable to avoid any potential overflow bugs.
- Added a function to check for `argc < 3`. This is a safety check which sanitizes the input of your program, which helps prevent crashes in the event of the user inputting invalid arguments for the application.

By adding these changes, we can avoid and fix the vulnerabilities detected by flawfinder. However, do note that these changes won’t be detected by flawfinder, since the program is coded to detect the mere presence of these vulnerabilities and doesn’t take into account any fixes we might have applied.